# SPARK, step by step: how the state evolves with each message

left: what the prover holds · middle: the message · right: what the verifier knows after it

| | PROVER | MESSAGE (transcript order) | VERIFIER |
|---|---|---|---|
| 0 | **Setup (once).** Public seed → gate pair $(t^0(p), t^1(p))$ for every tree node $p$ (keyed BLAKE3); checked setup: $C_{i_0}$ MDS, counter fixed. | canonical header → transcript | Recomputes the same gates from the seed. Validates $(n, k, i_0, s, g)$ once, recomputing $s$ itself. |
| 1 | **Encode.** $E_0(\omega) = F(\pi(\omega))$ for every leaf $\omega$: $2^{n+k}$ values in $\mathbb{F}$ ($2^{22}$ at $n$=20). Merkle tree → root $R_0$. | $R_0$ (32 bytes) → | Knows $R_0$: the prover is now *bound* to $E_0$ (it cannot change it without breaking the hash). |
| 2 | **Fold level 1:** at each node $p$, $E_1(p) = w^0 + \frac{z_1 - t^0(p)}{t^1(p) - t^0(p)}(w^1 - w^0)$. Size halves; values in $\mathbb{K}$; $E_1(p) = F(z_1, \pi(p))$. | ← $z_1 = H(\text{transcript}) \in \mathbb{K}$ | Knows $R_0, z_1$. Nothing to check yet. |
| 3 | **Levels** $2, \ldots, m$**:** same rule; each fold fixes one more variable: $E_j(p) = F(z_1, \ldots, z_j, \pi(p))$. No message for an uncommitted level. | ← $z_2, \ldots, z_m$ (round counter → distinct) | Knows $R_0, z_1, \ldots, z_m$. |
| 4 | **Commit layer** $m$ (schedule: every $m$ levels, $m$=3): Merkle root $R_m$ of $E_m$, grouped in blocks of $2^m$. | $R_m$ → | Knows $R_0, R_m$ and $z_1, \ldots, z_m$. Bound to $E_m$; the next challenges depend on $R_m$. |
| 5 | **Repeat** for all $n$ levels: fold with $z_j$; at $j \in \{3, 6, \ldots, 18\}$ send $R_j$. | ← $z_{m+1}, \ldots, z_n$ and $R_{2m}, R_{3m}, \ldots$ → | Knows all roots and all $z_j$: the point $z = (z_1, \ldots, z_n)$ is fixed. |
| 6 | **Last layer** $E_n$: one value per copy $\beta$ ($2^k$ values). Honest: all equal $y = F(z_1, \ldots, z_n)$. | $E_n$ in the clear ($2^k$ values of $\mathbb{K}$) → | Checks all entries equal; reads the **claimed value** $y$. Absorbs the whole $E_n$. |
| 7 | **Grinding:** searches a nonce whose hash has $g = 16$ leading zeros ($\approx 2^{16}$ hashes). | nonce → | Checks the $g$ zero bits (one hash). Adds $g$ bits to the query term. |
| 8 | Receives the query positions (derived from the full transcript). | ← $s = 407$ leaf positions $\omega_1, \ldots, \omega_s$ | Chooses the positions only *after* everything is bound. |
| 9 | **Openings:** for each $\omega_i$ and each committed layer, the block of $2^m$ values on its path, with Merkle path (deduplicated). | blocks + Merkle paths (bulk of the proof) → | **Checks per path:** Merkle paths against the roots; folds each block $m$ times with $z$ and its own gates; must hit the opened value one committed layer down, and finally $E_n(\beta) = y$. |

**Accept** $\Longrightarrow y = F^*(z_1, \ldots, z_n)$ for the polynomial $F^*$ bound by $R_0$.

Error terms: fold $n(N_1 + 1/\varepsilon)/|\mathbb{K}|$ and queries $2^{-g}(1 - \delta + n\varepsilon)^s$, each $\leq 2^{-192}$ (Thm 6.4; any schedule, Thm 6.7).

# Worked example over $\mathbb{F}_7$: the tables after each message

$F(x_1, x_2) = 3 + 2x_1 + 5x_2 + x_1x_2$, $n = 2$, $k = 1$ (two copies $\beta = 0, 1$), challenges $z_1 = 3$, $z_2 = 2$

**Gates (from the seed).**

Level of $x_2$ (one node per copy):
$t_2(0) = (1, 4)$, $t_2(1) = (3, 5)$

Level of $x_1$ (one node per $(\beta, \theta_2)$):
$t_1(0, 0) = (2, 4)$, $t_1(0, 1) = (1, 6)$
$t_1(1, 0) = (5, 4)$, $t_1(1, 1) = (5, 2)$

**Leaf point** $\pi(\beta, \theta_2, \theta_1) = \big(t_1^{\theta_1}(\beta, \theta_2),\ t_2^{\theta_2}(\beta)\big)$.

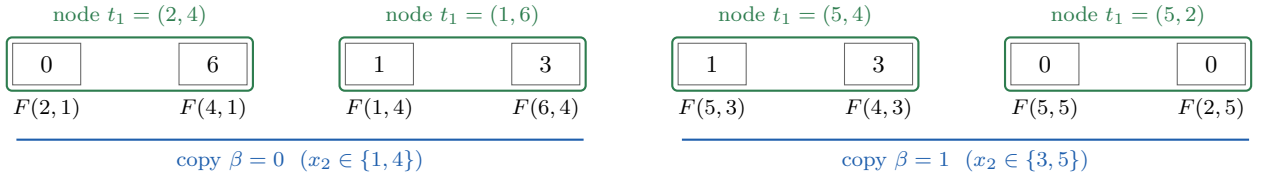
Example: $\pi(0, 1, 1) = (6, 4)$, so $E_0(0, 1, 1) = F(6, 4) = 59 \equiv 3$.

**What changed at each message**

| message | prover table | size |
|---|---|---|
| $R_0$ | $E_0(\omega) = F(\pi(\omega))$ | 8 |
| $z_1$ | $E_1 = F(z_1, \cdot)$ on gate pts | 4 |
| $z_2$ | $E_2 = F(z_1, z_2)$ | 2 |
| $E_2$ | sent in the clear | — |

At $n = 20, k = 2$: $2^{22} \to 2^{21} \to \cdots \to 4$ values; with $m = 3$ only every third table is committed.

## ① After the commitment: $E_0$ (8 values), root $R_0$ sent




## ② Verifier sends $z_1 = 3$; prover folds each $x_1$-node

$(0,0)$ : $0 + \frac{3-2}{4-2}(6-0) = 0 + 4 \cdot 6 = 3$ $\quad$ $(0,1)$ : $1 + \frac{3-1}{6-1}(3-1) = 1 + 6 \cdot 2 = 6$ $\quad$ $(1,0)$ : $1 + \frac{3-5}{4-5}(3-1) = 1 + 2 \cdot 2 = 5$ $\quad$ $(1,1)$ : $0$

Meaning: $E_1(0, 0) = F(3, 1) = 3$, $E_1(0, 1) = F(3, 4) = 6$, $E_1(1, 0) = F(3, 3) = 5$, $E_1(1, 1) = F(3, 5) = 0$ (all mod 7): $x_1$ is now fixed to $z_1$.




## ③ Verifier sends $z_2 = 2$; prover folds each $x_2$-node (one per copy)

copy 0: $3 + \frac{2-1}{4-1}(6-3) = 3 + 5 \cdot 3 = 4$ $\quad$ copy 1: $5 + \frac{2-3}{5-3}(0-5) = 5 + 3 \cdot (-5) = 4$



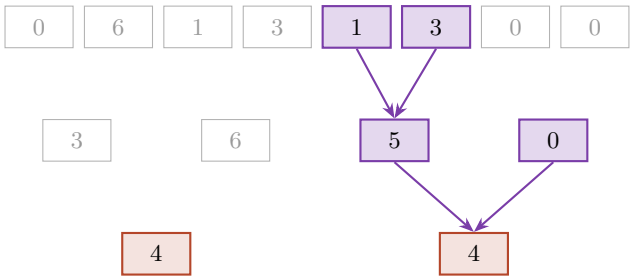

## ④ Prover sends $E_2 = (4, 4)$; verifier reads $y = 4$

Both copies agree, and indeed $F(z_1, z_2) = F(3, 2) = 3 + 6 + 10 + 6 = 25 \equiv 4$. Every layer keeps the same meaning:
$E_j(p) = F\big(z_1, \ldots, z_j,$ gate points on the path to $p\big)$, so the last layer is $F(z)$ in every copy.

**Why the fold is the right operation.** At a node with points $t^0, t^1$, the two children carry $w^b = u + t^b v$ for the split $G = u + x\,v$ of the current polynomial. The gate rule is the line through $(t^0, w^0)$ and $(t^1, w^1)$, so evaluating it at $z$ gives $u + z\,v = G(z)$: folding = substituting the challenge for one variable. The verifier can redo this on any opened pair, with gates it computes itself from the seed.

# The query phase, and the real sizes at each round

**Example: the verifier queries leaf $\omega = (\beta{=}1, \theta_2{=}0, \theta_1{=}1)$**




**Checks.**
1. Merkle paths of both pairs verify against $R_0$ and $R_1$.
2. Gate $t_1(1,0) = (5,4)$: $1 + \frac{3-5}{4-5}(3-1) = 5$ = opened $E_1(1,0)$ ✓
3. Gate $t_2(1) = (3,5)$: $5 + \frac{2-3}{5-3}(0-5) = 4$ $= E_2(1) = y$ ✓

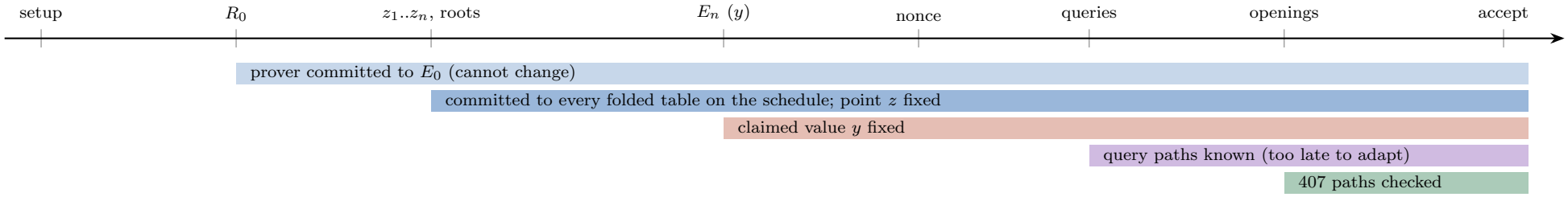
A cheating table must break the gate rule on many paths (distance of the code); each random path catches it with probability $\geq \delta$, so $s$ paths leave $(1-\delta)^s$.

**Production run:** $n = 20$, $k = 2$, $m = 3$, $s = 407$, $g = 16$

| round | message | table size | field | committed? |
|---|---|---|---|---|
| 0 | header | — | — | (transcript only) |
| 1 | $R_0$ | $2^{22}$ | $\mathbb{F} = \mathrm{GF}(2^{128})$ | yes |
| 2–4 | $z_1, z_2, z_3$ | $2^{21}, 2^{20}, 2^{19}$ | $\mathbb{K} = \mathrm{GF}(2^{256})$ | $2^{19}$: yes ($R_3$) |
| 5–7 | $z_4, z_5, z_6$ | $2^{18}, 2^{17}, 2^{16}$ | $\mathbb{K}$ | $2^{16}$: yes ($R_6$) |
| 8–19 | $z_7, \ldots, z_{18}$ | $2^{15}, \ldots, 2^4$ | $\mathbb{K}$ | $R_9, R_{12}, R_{15}, R_{18}$ |
| 20–21 | $z_{19}, z_{20}$ | $2^3, 2^2$ | $\mathbb{K}$ | no |
| 22 | $E_{20}$ | 4 values | $\mathbb{K}$ | in the clear |
| 23 | nonce | — | — | 16 zero bits |
| 24 | queries | 407 paths | — | — |
| 25 | openings | blocks of 8 | $\mathbb{F}$ / $\mathbb{K}$ | bulk of the proof |

roots sent: $R_0, R_3, \ldots, R_{18}$ (7 roots); challenges: 20
measured: proof 577 KiB (v0.9), prover 1.75 s, verifier 10.6 ms (v0.9.1)

## Who knows what, over time




**The order is the security.** Each challenge is a hash of everything sent before it (v0.9 transcript: header, $R_0$, then for each level the root if committed and the challenge, then $E_n$, nonce, queries). So the prover fixes each table before learning the challenge that folds it, fixes $y$ before learning the queries, and pays $2^{16}$ hashes per attempt to re-roll the queries.